Project: Optimizing Decision Trees Using Information Theory

Name: Thanh & Nam

Abstract

Decision trees are an essential tool in machine learning. They are among the most commonly used when you just can't do without them: simplicity, interpretability, and explanatory power for classification and regression tasks. We discuss in this paper the basic concepts of the Decision Tree and why it is so useful, like its structure, advantages, and disadvantages. Besides highlighting their role in reducing uncertainty and improving model efficiency, it investigates their properties in optimizing splits in terms of entropy and information gain. I present Feature selection as a critical step in building good decision trees and give examples of how truncating less important features can improve tree simplicity, prediction, and interpretability. Also addressed are overfitting, working with high dimensional data, and dealing with imbalanced data as well as how to prune to improve generalization and scalability. This study investigates these variables and offers experimental results confirming the suitability of decision trees for predictive modeling by pointing towards their construction and use optimized for real-world problems.

1. Introduction

Machine learning is loaded with fundamentals and decision trees that are easy to understand and interpret, and easy on the eyes. By using them for classification and regression tasks, they are widely used and their intuitive structure allows users to understand the reasoning behind making a prediction. A decision tree consists of nodes, each node being a decision given from a feature, and each branch is an outcome of that decision. Once the tree is large enough it becomes clear in what way it can be used to classify or predict the target variable.

A decision tree is a graph-like structure, which breaks the data into two subsets depending on predetermined attributes by breaking down systematically. It's a root node which is an initial decision or classification, it starts from it. At this point, the data is to be analyzed, and then a critical attribute is selected to create the first big cut generating the tree for the whole. At each point, as the data moves through the tree, it hits decision nodes — or intermediate points that represent something used to break the data further into smaller pieces based on some condition or rule. These nodes bless the tree for breaking down complex decision-making into smaller, more manageable pieces and push the data farther and farther toward finer outcomes. Finally, the process leads to the leaf nodes where final decisions or classifications happen at the terminal points of the whole tree and this all sums up the overall purpose of the tree; classify, predict, or recommend where they are used. Decision trees are great because they can form subtrees which is essentially another decision tree within the big picture. The subtrees improve the modularity

and the ability to adapt the decision-making process leading to localized adjustment of decisions and simplification of complicated problems. Overall, the hierarchical presentation of the decision tree—with root-to-leaf nodes and modular subtrees—provides a systematic and simple way of transporting an analysis of data as well as obtaining insight from it.

Although decision trees are very useful, they unfortunately also have problems. However, the complexity is now known for overfitting the training data, computational inefficiency when working with large datasets, and managing large high dimensional or imbalanced datasets. This is of critical importance to improving decision tree generalization, efficiency, and scalability. Compelling advantages characterize decision trees as an appealing approach to decision-making and predictive modeling. Such simplicity and interpretability are one of their most notable strengths as they imitate human decision-making processes, and are easy to understand. Decision trees are also very versatile and perform very reasonably well in classification and regression tasks. They also have another advantage only in terms of feature independence, since they do not require normalization or scaling of features, making preprocessing simpler. In addition, decision trees are well suited for nonlinear relationships which allow us to capture complex patterns between features and outcomes.

Although the above drawbacks stand out, decision trees are not free from faults. Overfitting is a major concern, and deep trees tend to overly memorize the training data, diminishing their ability to generalize to novel data. These trees are also unstable changes in the dataset may lead to very different tree structures. Decision trees may also be biased to such features that will have more levels (and hence have more categories), giving the features with many categories dominance on the structure and biasing the outcome. This last limitation shows us just how much care we need to put into the construction and regularization of our decision trees when trying them out in practice.

2. Entropy and Information Gain

The main aim of decision tree optimization is to make splits that decrease uncertainty most efficiently at each node. In this context, entropy is defined as a measure of randomness or impurity of a data subset at a given node. In this case, we define the entropy (H) of a set S as this is the number of elements that belong to class i out of S, where is the proportion of elements in class I over the set S and H is maximized when the data is spread evenly over all classes while is minimized when the dataset is composed only of a single class.

The amount of impurity or uncertainty in a dataset is quantified by entropy. Specifically in the context of decision trees, it refers to a measure of how combined or disorganized the classes in a dataset are. For a dataset S with n classes, entropy is calculated as:

$$H(S) = -\sum_{i=1}^{n} p_i \log_2 p_i$$

Here:

- Pi is the proportion of elements in class i.
- H(S) represents the entropy of the dataset S.

Entropy is maximum for maximum randomness (when the distribution of data over the classes is evenly distributed) and minimum when the data belongs entirely to one class (maximum order).

Example of Entropy Calculation:
Consider a dataset S with 10 samples:

- 6 samples belong to the class "Yes."
- 4 samples belong to the class "No."

The entropy of this dataset is:

$$p_{\text{Yes}} = \frac{6}{10}, \quad p_{\text{No}} = \frac{4}{10}$$

$$H(S) = -\left(\frac{6}{10}\log_2\frac{6}{10} + \frac{4}{10}\log_2\frac{4}{10}\right) = 0.97$$

For example, entropy is an important measure because it quantifies the impurity or uncertainty in a dataset. This helps us evaluate the quality of the dataset at a given node in the decision tree.

Behavior of Entropy in Decision Trees

- High Entropy: This means that we have a dataset with messy or diverse data in which no class is winning. For instance, if half the samples belong to "Yes" and the other half to "No," which is the maximum entropy, because the data is uniformly distributed across the two classes.
- Low Entropy: This corresponds to a homogeneous dataset where most samples belong to a single class. For example, if all samples are "Yes," the entropy is zero, as there is no uncertainty.

A decision tree splits to reduce entropy. The idea is to break the data into partitions of increasing homogeneity and have leaf nodes correspond to as pure as possible subsets (entropy = 0).

For this feature, the integer quantifying the reduction in entropy for dataset partitioning according to this feature is called Information Gain (IG). As lack of premature splitting corresponds to how good a particular attribute splits the data into mutually more homogeneous subsets that are. The information gain for a split on attribute A is given by: That is, if we have for attribute A the value v we have that, By choosing the best possible information gain at each step, we determine the best splits and form an efficient decision tree that separates classes in as differentiating a way as possible.

After a split it gives the reduction in the entropy. For an attribute A, splitting S into subsets Sv, the formula is:

$$IG(S, A) = H(S) - \sum_{v} \frac{|S_v|}{|S|} H(S_v)$$

Where:

- IG(S, A): Information gain for dataset S when split on attribute A.
- H(S): Entropy of the original dataset S.
- Sv: Subset of S where attribute A has value v.
- H(Sv): Entropy of the subset Sv.

The goal of a decision tree algorithm is to maximize information gain at each split, ensuring that the dataset is divided into subsets with the greatest reduction in entropy.

Example of Information Gain Calculation:
Suppose we have a dataset SSS with entropy H(S)=0.97, consisting of 10 samples.

- Split S on attribute A, producing two subsets:
    - S1: 4 samples (3 "Yes," 1 "No"), with H(S1)=0.8.
    - S2: 6 samples (3 "Yes," 3 "No"), with H(S2)=1.0.

The weighted average entropy after the split is:

$$H_{\text{split}} = \frac{4}{10} \cdot H(S_1) + \frac{6}{10} \cdot H(S_2) = 0.92$$

The information gain from this split is:

$$IG(S, A) = H(S) - H_{\text{split}} = 0.97 - 0.92 = 0.05$$

In this example, attribute A provides a small reduction in entropy, indicating that it is not a highly effective feature for splitting.

Entropy and Information Gain in Decision Tree Construction

The interplay between entropy and information gain drives the construction of decision trees. At each step:

1. Evaluate Entropy: This allows us to calculate the entropy of the dataset at the current node to determine its impurity.
2. Calculate Information Gain: For each candidate attribute, compute the information gain associated with splitting the dataset based on that attribute.
3. Select the Best Attribute: Split the current node based on the attribute that has the highest information gain.

This iterative procedure constructs the decision tree so that the entropy of the subsets that each node determines is minimized and the homogeneity of subsets determined by each node is maximized.

Key Insights on Entropy and Information Gain

1. Entropy as a Measure of Impurity: If we think about entropy as a measure of random (or uncertainty) in a dataset, then the greater the entropy in the dataset, the more we believe there is no information that we can extract from the data. A dataset of zero entropy is a perfectly pure dataset and a dataset of maximum entropy is a perfectly mixed dataset.
2. Information Gain as a Metric for Improvement: The amount of difference in entropy achieved when the dataset is split is called information gain. The higher the information gain indicates a more effective split.
3. Guiding Tree Construction: Decision trees iteratively maximize information gain such that the tree is at its maximum discriminate power between classes.

Practical Benefits of Entropy and Information Gain

1. Improved Predictive Accuracy: Decision trees effectively predict by focusing on features that will maximize the gain of information.

2. Streamlined Models: The tree is a simpler model, resulting in a more interpretable model, as attributes with low information gain are excluded.
3. Efficiency: By reducing entropy at each step, we create more homogeneous subsets, speeding both tree construction and subsequent predictions.

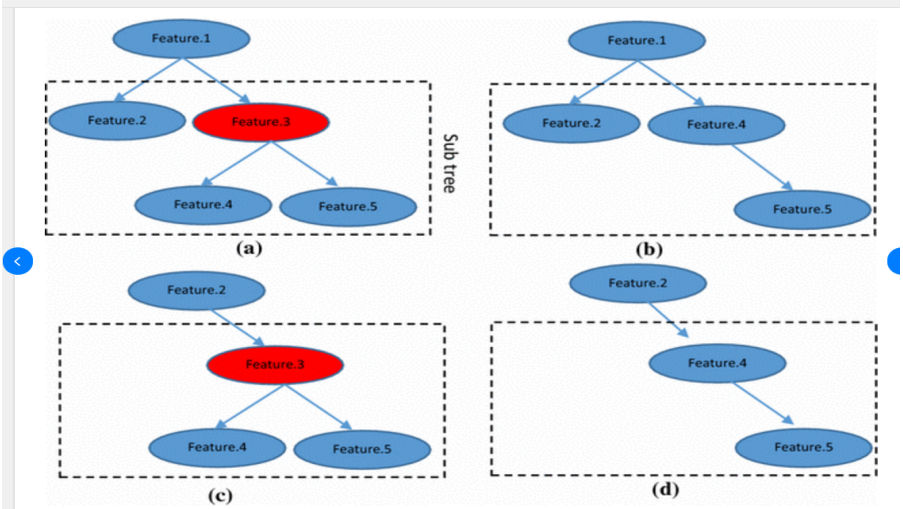
There are two key ideas on which to build decision trees: Entropy and information gain. The second one is that entropy is a measure of the information contained in the dataset and information gain is a method we use to quantify the amount of uncertainty that can be relieved, by choosing the splits that have the highest amount of information. By combining these two concepts, we bring simplicity and predictive power together, the same thing decision trees were designed for, and they are a good tool for classification and decision-making tasks. By systematically reducing entropy and maximizing information gain, decision trees achieve their ultimate goal: How to get efficient, interpretable models that do well out of sample.

3. Feature Selection

Optimizing decision trees is particularly relevant when the data space is high dimensional; feature selection is an essential step in practicing it. Feature selection is interpreted as pillars of interpretability, efficiency, and computational feasibility and cannot be thought of as a cakewalk task of either building accurate but inefficient interpretable models or computationally feasible ones. By selecting the most relevant features, we can achieve multiple objectives: He shows that they decrease computational costs, help models generalize better, and are interpretable. These advantages imply that selecting features is a vital step while constructing decision trees.

One of the most commonly used metrics to select features is information gain which is the quantity by which entropy of the data decreases when we split the data on some particular feature. We consider features with higher information gain as more informative ones and features that scarcely contribute to entropy reduction exclusion. This results in an attribute set that is optimally suited to decision tree construction, derived from a systematic ordering and pruning of features. As the model eliminates the baggage of attributes that are necessary but are not helpful to complexity and performance, they are cleaned through focus based on the most important ones.

This image illustrates feature selection in a decision tree through the elimination of less significant features Let's break it down step by step based on the labeled subfigures (a, b, c, d):



Feature selection with DT method. a Decision tree. b Eliminate the feature.3. c Eliminate the feature.1. d Eliminate the feature.3

(a): Original Decision Tree

- This represents the initial, fully expanded decision tree, which includes all available features. The tree's structure is as follows:
- It contains all the features:
    - Feature 1 serves as the root node, determining the first split.
    - Feature 2 and Feature 3 occupy subsequent decision points within the tree.
    - Leaf nodes are determined by Feature 4 and Feature 5, which represent the outcomes or final decision points.

While this tree captures all the information in the dataset, it may include redundant or non-informative features. These unnecessary features increase the tree's complexity, potentially leading to issues such as overfitting and reduced interpretability.

(b): Eliminating Feature 3

- In this stage, Feature 3 is identified as contributing minimally to the decision-making process and is therefore removed. The tree is then restructured to adapt to the elimination of this feature:
- After elimination, the decision tree is adjusted:
    - Feature 1 still serves as the root.
    - Feature 2 connects directly to the leaf nodes represented by Feature 4 and Feature 5.

This adjustment simplifies the tree by reducing its size and complexity. Importantly, the removal of Feature 3 does not significantly affect the tree's predictive power, demonstrating the potential for feature selection to streamline models without compromising accuracy.

(c): Decision Tree Focusing on Feature 2 and Feature 3

- In this variation, Feature 1 is excluded instead. The decision tree is restructured to focus on the remaining features:
    - Feature 2 becomes the new root node.
    - Feature 3 takes the role of the subsequent decision point, influencing the pathways to the leaf nodes.

This adjustment might be relevant in scenarios where Feature 1 offers lower information gain compared to Feature 2 and Feature 3, or when Feature 1 is deemed irrelevant due to domain-specific considerations. The result is a shift in focus that highlights the importance of tailoring feature selection to the specific characteristics and requirements of the dataset.

(d): Final Simplified Tree

- In the final stage of optimization, both Feature 1 and Feature 3 are eliminated. The resulting tree is the most streamlined version:
- The decision tree is further simplified:
    - Feature 2 becomes the sole decision-making node.
    - Leaf nodes are directly determined by Feature 4 and Feature 5.

This minimalist representation excludes all extraneous features that would unnecessarily complicate decision-making pathways while preserving the important high-level decision-making pathways needed by programmers. The tree focuses on the most critical attributes resulting in maximum interpretability and computational efficiency.

Key Takeaways from Feature Selection in the Diagram

1. Goals of Feature Selection:
    - Simplify Complexity: Shrink the number of decisions and splits they have.
    - Retain Predictive Power: Make sure that this won't degrade the model's accuracy and generalization ability.
2. Elimination Criteria: Entropy reduction and accuracy of the overall model are considered, and features are removed based on their contribution towards reducing the entropy of the model. Low information gain features are considered redundant and omitted from the model.
3. Benefits of Feature Selection:

    - Reduces overfitting: By focusing on the most informative features, the model avoids capturing noise or irrelevant patterns in the data.
    - Improves Interpretability: Simplifying the trees in this way makes these trees easier to understand and explain, especially in fields where you need transparency, like healthcare and finance.
    - Enhances Computational Efficiency: The fewer features, the fewer computational resources needed to train, and also reduced computational resources required for inference.
4. Process Illustrated:
    - How decision trees can be systematically refined by iterative removal of less significant features is shown. Each step in the process highlights a key principle of feature selection: keeping it simple yet having performance.

Effectively, the visualization shows how the feature selection affects the decision trees. Starting from the full feature expanded version, moving towards the final streamlined version, every step points out the virtue of focusing only on the most important feature. Feature selection helps improve interpretability and reduces redundancy, while at the same time improving computational efficiency, resulting in a powerful way to optimize decision trees for real-world applications.

Pruning Strategies

Pruning is a critical technique in machine learning used to optimize decision tree models, improving generalization and preventing overfitting. Pruning strategies are almost indispensable in generating a high-quality decision tree, given that datasets in the real world are enormous. By definition, Pruning removes nodes or branches that do not provide significant predictive power to the model, thus improving its ability to generate unseen data more efficiently and precisely.

The concept of pruning and feature selection shares the same idea of reducing complexity and enhancing generalization ability, but there are some key differences.

Feature selection occurs before the model is built. It aims to identify the most relevant input (features) and remove all those that are redundant, therefore reducing the dimensionality of the data and simplifying the modeling process. Whenever we build a decision tree, we start by determining a list of input features. Features selection focuses on minimizing that list yet still guarantees the performance.

In contrast, pruning is specific to decision trees and happens either during or after the tree construction. While feature selection deals with the input data itself, pruning refines the internal structure of the decision tree. When a branch is pruned, it eliminates a decision point based on a specific feature at that particular node. However, this does not prevent the feature from being used elsewhere in the tree. The feature may still be involved in splits at other nodes in the tree

that remain after pruning.

In summary, feature selection deals with the list of input features, possibly removing the redundant features permanently from getting included in the model. Pruning aims to reduce tree structure complexity by removing a specific uninformative branch(es) at a specific point in the tree, even if all features are retained (as long as the features exist somewhere else in the tree). Both strategies complement each other, but should not be mistaken as one.

Type of Pruning

There are 2 main types of pruning - pre-pruning (also known as early stopping) and post-pruning (also known as node reducing). Each approach addresses the problem at different stages of generating a tree, with different methods and benefits.

Cross-validation

Before jumping into the different strategies of pruning, it is important to understand the cross-validation technique. Cross-validation is used to assess the performance of a machine-learning model by splitting the dataset into multiple subsets. We then train and test the model on different combinations of these subsets. By doing so, we aim to ensure that the model is not overfitting to the training data and predicts future data correctly to a great extent.

The most common method is K-fold cross-validation. It is simple yet effective and is universally used in many researches. There are 3 steps:

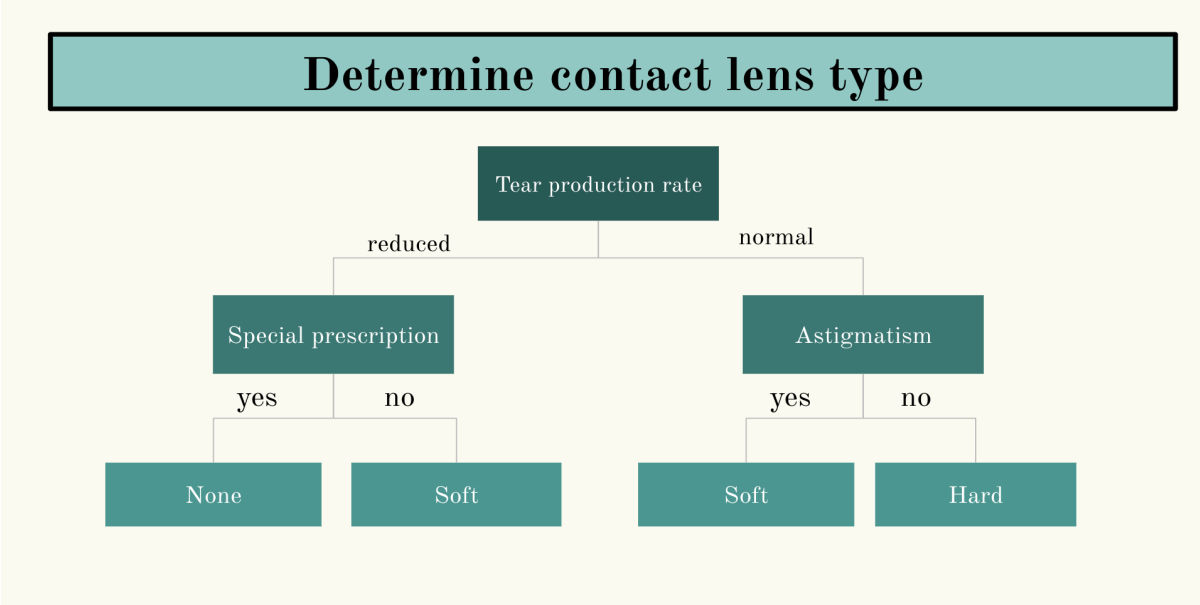
1. Split the dataset into k equal parts (nearly equal is fine): If you use 10-fold cross-validation, the dataset is divided into 10 parts.
2. Train the model k times:
    - In each iteration, one of the k subsets is used as the validation set, while the remaining k-1 subsets are used for training.
    - This process is repeated for each fold, so the model is trained and tested k times.
3. Evaluate model performance:
    - After all iterations, the performance of the model is averaged across the k-test sets to get an overall estimate of its performance.

Pre-pruning (Early stopping)

A tree is “pre-pruned” by halting its construction early when reaching a certain threshold. It is important to determine the correct constraint to ensure your decision tree remains well-fitted and informative enough. The methods for figuring out this threshold vary with different pre-pruning techniques. The most common ones are:

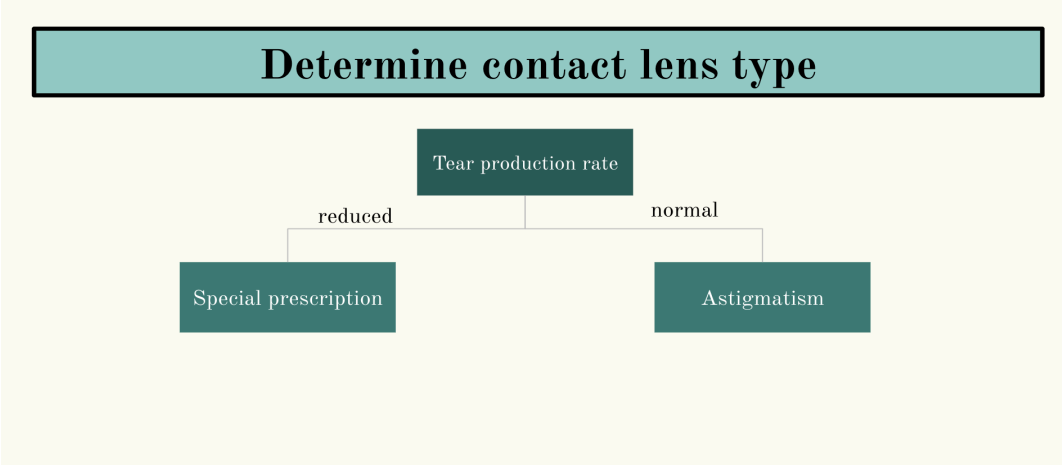
1. Maximum depth
    - Set a maximum depth (threshold) for the tree to prevent it from growing too complex. This helps avoid overfitting by limiting how far the tree can split.
    - We tested different depth levels using cross-validation to find out the most informative depth
2. Minimum samples per leaf
    - Set a minimum number of samples required to split a node.
    - We test different values of minimum leaf sizes (e.g., 1, 2, 5, etc.) via cross-validation.
3. Minimum samples per split
    - Specifies the minimum number of samples required to make a split at any node.
    - If dataset is small, you can even increase this parameter to limit the number we have to split
    - Use cross-validation to figure out the right parameter. Start with the lowest value and work your way up
4. Maximum Leaf Nodes
    - This constraint restricts the number of leaf nodes in the tree
    - Based on datasets, choose a proper number of nodes then test it using cross-validation
    - For example, a dataset of 10 should start with 3 nodes, but datasets of 1000 should start using 30 nodes

To be more clear with the process of post-pruning, I propose an example below.



This decision tree was created to figure out the most suitable contact lens types based on customer's physical condition. The depth of the tree is 3. This is an example of a tree fully generated without any pruning or any interaction with the datasets.

Now we will apply the maximum depth methods. To speed up the explanation, let's assume we perform cross-validation and decide that a maximum depth of 2 is informative enough to give out contact lens type to the customer without any repercussions. Then during the process of growing this decision tree, we grow until we reach the depth of 2, and we stop there. The tree will just become this:



Benefits and Drawbacks of Pre-pruning

Pre-pruning is computationally efficient because it avoids building many unnecessary parts of the tree. However, its primary drawback is that it may stop the growth of the tree too early, potentially missing important splits that could have improved the model's performance if we continue. This can lead to underfitting, which means the model is too simple to capture the underlying patterns in the data. In the real world, we almost always desire an accurate analysis over runtime, which implies that post-pruning is generally preferred over pre-pruning due to its reliable correctness. Pre-pruning also performs poorly when exposed to new data.

Post-pruning

In contrast to pre-pruning, post-pruning occurs after the decision is fully built. This means we let the trees grow to their maximum depth, evaluate the result, and then start trimming unnecessary branches. This process of tree simplification is more complicated than pre-pruning and demands high-quality processors in most cases. The most common methods for post-pruning are:

1. Cost-Complexity Pruning (CCP):
    - This method assigns a price to each subtree primarily based on its accuracy and complexity, then selects the subtree with the lowest fee.

a. Complexity measure: For each subtree in the tree, calculate its cost. The cost is determined by a combination of:
    - Accuracy: The error or misclassification rate of the subtree.
    - Tree Size: The number of nodes (or splits) in the subtree.
b. Pruning Criteria:
    - The cost of the subtree is adjusted by a factor called α (alpha). This parameter penalizes larger trees, effectively controlling how much complexity is acceptable.
    - The cost function is typically calculated as:

$$\text{Cost of subtree} = \text{Error rate} + \alpha \times \text{Size of subtree}$$

    - As α increases, the algorithm will prefer smaller trees because the penalty for complexity (number of nodes) becomes larger.
c. Step by step:
    - Grow the trees fully
    - Calculate the cost for each subtree at different levels while growing
    - For each value of α, prune the subtree that results in the lowest cost.
    - Select the subtree with the lowest total cost.

d. Benefits and drawbacks
    - Benefits: a good balance between complexity and accuracy
    - Drawbacks: choosing α can be challenging

2. Reduced Error Pruning:
    - Removes branches that do not significantly affect the overall accuracy.
    a. Step by step:
        - Grow the tree fully
        - Start from the leaves and work upwards towards the root.
        - At each node, evaluate the subtree rooted at that node.
        - Replace the entire subtree with a single leaf node that represents the most frequent class (or the mean, if regression).
        - If replacing the subtree with a leaf node does not result in a loss of accuracy on the validation set, prune the node.
        - Continue this process recursively until no further pruning can improve or maintain the accuracy.

    b. Benefits and drawbacks
        - Benefit: simple and intuitive, does not require any cost function
        - Disadvantages: can be very expensive if there are too many nodes to prune. Also requires data validation sets which reduces training data

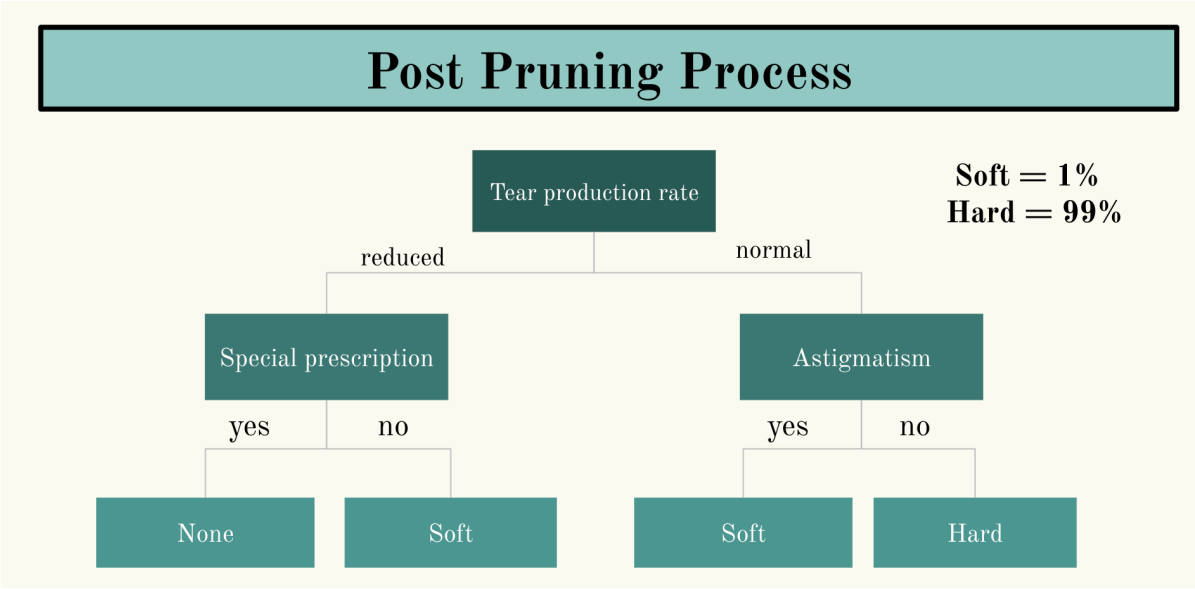
3. Minimum Impurity Decrease:
    - Prunes nodes if the decrease in impurity (Gini impurity or entropy) is beneath a certain threshold.
    - If it's below the threshold, it has affected the performance of the trees, so it is advisable not to prune
    a. Step-by-step guide
        - Grow the decision tree to its full size.
        - After growing the tree, evaluate each node split. At each node, check the reduction in impurity (e.g., Gini impurity or entropy) resulting from the split.
        - If the decrease in impurity is smaller than a defined threshold (a predefined minimum impurity decrease), prune the split and replace the node with a leaf node.
    b. What is Gini impurity?
        - Gini impurity is a measure used in decision tree algorithms to quantify a dataset's impurity level or disorder.
        - In binary classification problems, it assesses the likelihood of an incorrect classification when a randomly selected data point is assigned a class label based on the distribution of classes in a particular node.
        - It ranges from 0 to 0.5, where 0 indicates a perfectly pure node (all instances belong to the same class), and 0.5 signifies maximum impurity (an equal distribution of classes).
        - In decision trees, it aids in selecting the optimal split by identifying features that result in more homogeneous subsets of data, ultimately contributing to the creation of accurate and reliable predictive models
    c. Benefits and drawbacks
        - Benefits: Highly precise due to entropy of Gini, preserving the predictive power
        - Drawbacks: Less intuitive and threshold may be too high which leads to underfitting

4. Minimum Leaf Size:
    - Remove leaf nodes with fewer samples than a specified threshold.
    a. Step-by-step:
        - Grow the trees to their full size
        - Evaluate the size of the leaf nodes.
        - If the number of samples in a leaf node is smaller than a defined threshold (e.g., fewer than 3 samples), prune that leaf node and replace it with another node representing the most frequent class or mean (in the case of regression).

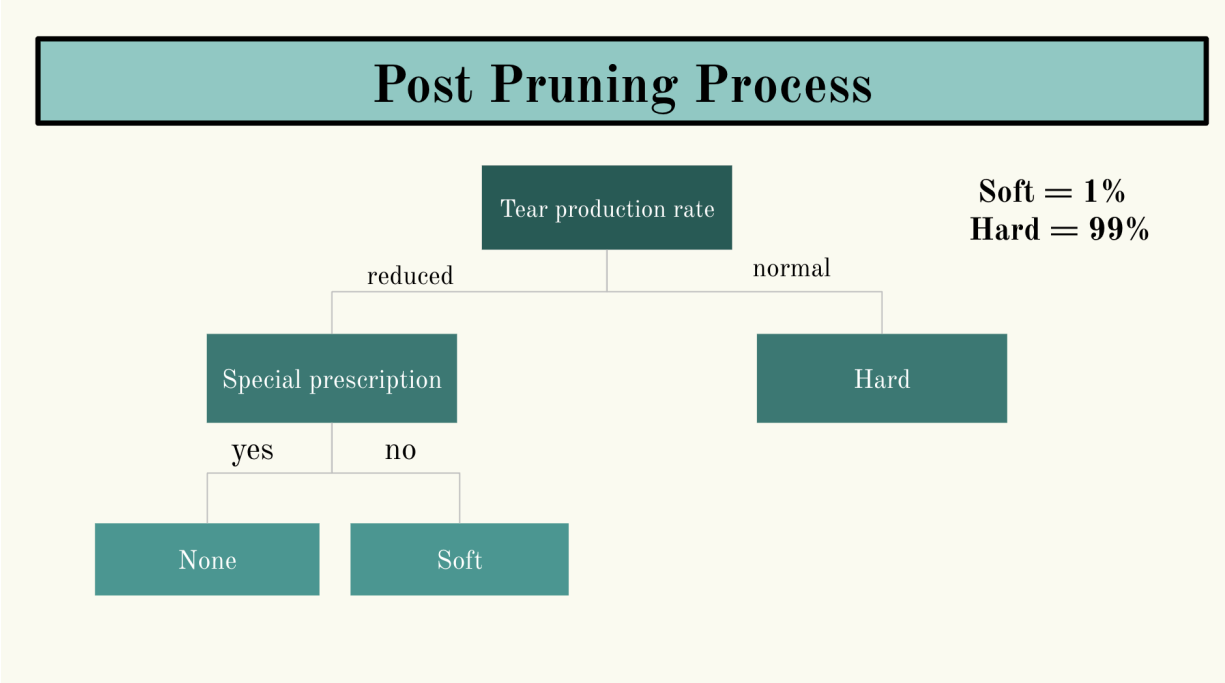
- Small samples of leaf nodes are likely to be noise or outliers in our datasets

b. Benefits and drawback
   - Benefits: Simple to implement, much more robust and generalizable
   - Drawbacks: May require tuning to find the best threshold

5. Detailed example
   - Let's try to apply "Reduced Error Pruning" to our previous example of contact lens
   - Assuming that for Astigmatism, the hard choice account for 99% of the model accuracy, while soft choice account for only 1%



- With that in mind, we can prune the branches, and replace the Astigmatism with the leaf nodes Hard, and the model accuracy will be pretty much the same



Handling Imbalanced Data

1. What is Imbalanced Data?

Imbalanced Data is a very common problem in machine learning. It is a condition where classes are not represented equally or in other words, one class has more instances than the others. This condition can cause several problems such as the model cannot classify the minority class, you cannot use accuracy as the performance metrics

2. Solution
    a. Oversampling
        - Oversampling is a technique to increase the number of instances of minority class so that it equals the majority class.
        - Copying the number of instances of the minority class.
        - One method that you can use to do Oversampling is SMOTE (Synthetic Minority Oversampling Technique).
        - SMOTE is an oversampling technique that creates synthetic examples rather than duplicating minority class samples. It generates new samples by interpolating between existing minority class instances. For each minority instance, SMOTE identifies its k-nearest neighbors and creates synthetic samples by combining the feature vectors of the instance and its neighbors.
    b. Undersampling

- Undersampling on the opposite is a technique to reduce the number of instances from the majority class so that it equals the minority class.
- The method that you can use is Random Under Sampling.
- Random under-sampling reduces the number of majority class samples by randomly removing instances. This technique aims to balance the class distribution by discarding some of the majority class examples.

Conclusion

Classification and regression with decision trees are powerful tools: Simple, interpretable, and efficient. However, one will not be able to fully unlock their potential to solve common problems such as overfitting or computational inefficiency or reduce generalization. It is concerned with letting foundational concepts like entropy and information gain applied to trees that optimize splits, as well as feature selection and pruning approaches to simplify tree structure. Refining decision trees depends on pruning strategies: pre-pruning and post-pruning. Pre-pruning is simply blocking tree growth earlier that is based on constraints such as limits per depth or minimum samples per split post pruning is much subtler and involves evaluating and removing branches created after tree construction is finished. These strategies eliminate the problem of overfitting while keeping predictive power for decision tree generalization. Cost complexity pruning and minimum impurity decrease ensure that the final tree is good in complexity and good in accuracy each in its way, which it is not to real-world applications. Secondly, the more we learn about metrics like Gini impurity, the more we can grasp its usefulness in feature selection and node splitting in the case where the data is high dimensional or imbalanced. Through these approaches, we systematically optimize decision trees to produce models that, in addition to being accurate, are interpretable, scalable, and efficient, a highly desirable property for use in machine learning. Finally, combining theoretical principles, computational strategies, and domain specific requirements is necessary for optimizing decision trees. Using these optimization methods, decision trees can be used to solve diverse predictive modeling challenges with effective solutions to complex real world problems.

Citation:

1, GeeksforGeeks. "Decision Tree Introduction with Example." Accessed December 15, 2024. https://www.geeksforgeeks.org/decision-tree-introduction-example/.
2, Meila, Marina, and Michael I. Jordan. "Application of Information Theory to the Construction of Efficient Decision Trees." Accessed December 15, 2024. https://typeset.io/pdf/application-of-information-theory-to-the-construction-of-18sab7u7up.pdf.

3, Safavian, Soroush R., and David Landgrebe. "A Survey of Decision Tree Classifier Methodology." *Pattern Recognition* 21, no. 3 (1991): 241–57. Accessed December 15, 2024. https://www.sciencedirect.com/science/article/pii/S1042814305800202.
4, Ravindran, Shubham. "Decision Tree for Classification: Entropy and Information Gain." Medium. March 22, 2021. Accessed December 15, 2024. https://medium.com/codex/decision-tree-for-classification-entropy-and-information-gain-cd9f99a26e0d.
5, Pramod, Om. "Decision Trees." Medium. July 14, 2021. Accessed December 15, 2024. https://medium.com/@ompramod9921/decision-trees-6a3c05e9cb82.
6, GeeksforGeeks. "Pruning Decision Trees." Accessed December 15, 2024. https://www.geeksforgeeks.org/pruning-decision-trees/.
7, Analytics Vidhya. "What is Gini Impurity?" Accessed December 15, 2024. https://www.analyticsvidhya.com/articles/gini-impurity/#h-what-is-gini-impurity.
8, 160 Shelf. "Decision Tree on Imbalanced Dataset." Medium. Accessed December 15, 2024. https://medium.com/@160shelf/decision-tree-on-imbalanced-dataset-f1575414a6c2.